



UNIVERSITÀ DI PISA

UNIVERSITÀ DI PISA

*Department of Computer Science
Master's in Data Science & Business Informatics*

Constrained Clustering in the L_2 Norm: Heuristic and Exact Methods

Project Report

(P) is the Constrained Clustering Problem in the L_2 norm: find k centroids for the clusters and assign each of the n inputs to exactly one centroid so as to minimise the sum of the L_2 distances between each point and the assigned centroid, with the constraint that no centroid must be assigned more than a pre-determined number m ($\geq \lceil n/k \rceil$) of points.

(A1) is a heuristic approach obtained by suitably modifying the k -means one.

(A2) is the exact approach of applying a general-purpose solver to an appropriate formulation of the problem.

Submitted To:

Prof. Antonio Frangioni

Submitted By:

Federica Braghin (698156)
Nicholas Vannucci (691095)

**Group 66
Academic Year 2025/26**

Abstract

In this report we address the constrained clustering problem under the L_2 norm, corresponding to the Euclidean distance. The objective is to determine k centroids and assign n data points to exactly one centroid, minimising the total sum of squared L_2 distances between each point and its assigned centroid. The problem includes a capacity constraint: no centroid may be assigned more than $m \geq \lceil n/k \rceil$ points, which distinguishes it from the classical unconstrained k -means problem. The data $x_i \in \mathbb{R}^d$ are assumed finite and belong to a hyper-rectangle $\prod_{\ell} [L^{\ell}, U^{\ell}]$ determined by the data themselves.

We explore two main approaches: (A1) a heuristic method based on a modified version of k -means, which handles the capacity constraint via a constrained assignment problem solved at each iteration using Minimum Cost Flow, and (A2) an exact method formulated as a convex Mixed-Integer Quadratic Program (MIQP), which guarantees the optimal solution through an exact reformulation with auxiliary variables $z_{ij}^{\ell} = a_{ij} \cdot c_j^{\ell}$, but becomes computationally expensive as the problem size grows.

This report analyses both methods from a theoretical and experimental perspective, with particular attention to the theoretical foundations of A1, its convergence behaviour and empirical performance, as well as the computational limits of A2 on instances of increasing size.

Contents

1	Introduction	3
1.1	The Constrained Clustering Problem (P)	3
1.2	Heuristic Approach: Modified k -means (A1)	5
1.2.1	Centroid Update Step	5
1.2.2	Constrained Assignment Step	6
1.3	Exact Approach: L_2 Clustering as a MIQP (A2)	7
1.3.1	Mixed-Integer Quadratic Programming (MIQP)	7
1.3.2	Characteristics	7
1.3.3	Use Cases	7
1.3.4	Strengths	7
1.3.5	Weaknesses	7
1.3.6	Decision Variables and Formulation	8
2	Optimization Algorithms	10
2.1	Heuristic Approach (A1)	10
2.1.1	Algorithm 1: Modified k -means with Capacity Constraint	11
2.2	Explanation of Algorithm 1	11
2.2.1	Initialization	11
2.2.2	Constrained Assignment Step	11
2.2.3	Centroid Update Step	12
2.2.4	Convergence and Final Selection	13
2.2.5	Computational Complexity	13
2.3	Exact Approach (A2)	14
2.3.1	Algorithm 2: MIQP for Constrained Clustering	14
2.4	Explanation of Algorithm 2	14
2.4.1	Model Construction	14
2.4.2	Resolution with Gurobi and Branch-and-Bound	15
2.4.3	Solution Extraction and Optimality Gap	15
2.4.4	Computational Complexity	16
3	Experiments	17
3.1	Experimental Setup and Dataset	17
3.2	Experiments on Synthetic Data	19
3.2.1	Monotone Decrease of A1's Objective	19
3.2.2	A1 Convergence: Iterations and Time as a Function of n	19
3.2.3	A2 Scalability	20
3.2.4	GAP between A1 and the Global Optimum	22
3.2.5	Required Number of Restarts	22
3.3	Effect of the Capacity Constraint m	24
3.4	Discussion	25
4	Conclusions	27

1 Introduction

This chapter introduces the constrained clustering problem addressed in this project and the theoretical foundations of the two proposed approaches. Section 1.1 provides the mathematical formulation of the problem and analyses the key properties of the objective function. Section 1.2 describes the heuristic approach (A1), based on a modified version of k -means with a constrained assignment step. Section 1.3 presents the exact approach (A2), formulated as a Mixed-Integer Quadratic Program (MIQP).

1.1 The Constrained Clustering Problem (P)

Given a set of n points $x_i \in \mathbb{R}^d$ for $i = 1, \dots, n$, the constrained clustering problem under the L_2 norm aims to partition the points into k clusters by determining the optimal positions of the centroids, using the squared Euclidean distance as a dissimilarity measure. Unlike the classical k -means problem, a capacity constraint is introduced: no centroid may be assigned more than $m \geq \lceil n/k \rceil$ points [1]. Since the data are finite, for each dimension ℓ we define $L^\ell = \min_i x_i^\ell$ and $U^\ell = \max_i x_i^\ell$: the points belong to the hyper-rectangle $\prod_\ell [L^\ell, U^\ell]$. It can be easily verified that the optimal centroids must also belong to it: being weighted averages of points in $\prod_\ell [L^\ell, U^\ell]$, they belong to it by convexity of the hyper-rectangle.

The problem is formulated as:

$$\min \sum_{i=1}^n \sum_{j=1}^k a_{ij} \|x_i - c_j\|^2 \quad (1)$$

where:

- $x_i \in \mathbb{R}^d$ denotes the i -th data point, with $i \in \{1, \dots, n\}$;
- $c_j \in \prod_\ell [L^\ell, U^\ell]$ denotes the centroid of cluster j , with $j \in \{1, \dots, k\}$;
- $a_{ij} \in \{0, 1\}$, for all i, j , is the binary assignment variable: it equals 1 if point x_i is assigned to centroid c_j , and 0 otherwise.

subject to:

$$\sum_{j=1}^k a_{ij} = 1, \quad \forall i \in \{1, \dots, n\} \quad (2)$$

$$\sum_{i=1}^n a_{ij} \leq m, \quad \forall j \in \{1, \dots, k\} \quad (3)$$

Constraint (2) ensures that every point is assigned to exactly one centroid. Constraint (3) imposes the capacity limit, distinguishing this formulation from the classical unconstrained k -means problem [2]. Finding the exact solution of this problem is NP-hard, even in the unconstrained case with only two clusters [3]. The assignment variables a_{ij} are binary and the number of possible assignments grows exponentially with n , making complete enumeration infeasible even for moderately sized instances. This motivates the use of a heuristic approach (A1) and an exact solver (A2).

The objective function (1) has the following key properties:

- **Differentiability.** Unlike the L_1 norm, the function $\|x_i - c_j\|^2$ is differentiable everywhere with respect to c_j . This allows us to compute the gradient:

$$\nabla f(c_j) = -2 \sum_{i=1}^n a_{ij} (x_i - c_j) \quad (4)$$

and the Hessian:

$$\nabla^2 f(c_j) = 2 \left(\sum_{i=1}^n a_{ij} \right) I \quad (5)$$

where I is the $d \times d$ identity matrix. Since $\sum_{i=1}^n a_{ij}$ is the number of points assigned to cluster j , which is always positive, the Hessian is positive definite everywhere — implying strict convexity of the function with respect to c_j , as discussed in the next property. This differentiability allows us to apply Theorems 3.1 and 3.3 [4] to characterise the optimal centroid analytically.

- **Convexity of the quadratic part.** When the assignments a_{ij} are fixed, the problem of finding the optimal centroid of each cluster j becomes:

$$\min_{c_j} f(c_j) = \min_{c_j} \sum_{i=1}^n a_{ij} \|x_i - c_j\|^2 \quad (6)$$

Due to the differentiability discussed above, the Hessian (5) is positive definite everywhere, so the function is strictly convex over all of $\mathbb{R}^d$.

Applying Theorem 3.1 [4] — first-order necessary condition — we set the gradient (4) equal to zero:

$$\nabla f(c_j) = -2 \sum_{i=1}^n a_{ij} (x_i - c_j) = 0 \quad (7)$$

Expanding:

$$\sum_{i=1}^n a_{ij} x_i - c_j \sum_{i=1}^n a_{ij} = 0 \quad (8)$$

which gives:

$$c_j^* = \frac{\sum_{i=1}^n a_{ij} x_i}{\sum_{i=1}^n a_{ij}} \quad (9)$$

which is exactly the **mean** of the points assigned to cluster j .

Applying Theorem 3.3 [4] — second-order sufficient condition — since $\nabla f(c_j^*) = 0$ and $\nabla^2 f(c_j^*) \succ 0$, the stationary point c_j^* is a local minimum. However, since the Hessian is positive definite *everywhere* — not only at c_j^* — the function is globally strictly convex, and therefore every local minimum coincides with the **unique global minimum** [4].

This convexity guarantees two things: that the centroid update step in A1 always finds the exact solution, and that A2 can be formulated as a MIQP through an exact reformulation with auxiliary variables z_{ij}^ℓ [4].

- **Global non-convexity.** The complete problem (1), considered jointly in the variables a_{ij} and c_j , is not globally convex. The binary nature of the assignment variables introduces multiple local minima, and A1 does not guarantee finding the global optimum. This motivates the use of A2 as an exact approach.
- **Lipschitz Continuity.** Since the data are finite and belong to the hyper-rectangle $\prod_{\ell}[L^{\ell}, U^{\ell}]$, the objective function is Lipschitz continuous with respect to the centroids, with constant bounded by $L \leq 2n\sqrt{d} \max_{\ell}(U^{\ell} - L^{\ell})$. Small perturbations of the centroids therefore produce controlled variations in the objective function, guaranteeing the stability of the optimisation process and the existence of a reachable global minimum.

1.2 Heuristic Approach: Modified k -means (A1)

Heuristic techniques address complex problems using practical strategies, trade-offs and empirical rules rather than guaranteeing the optimal solution [5]. A1 is a modified version of the classical k -means that introduces the capacity constraint m : no centroid may receive more than $m \geq \lceil n/k \rceil$ points. The algorithm iteratively alternates two steps — constrained assignment and centroid update — until convergence. Since the result depends on the initialisation, r independent initialisations are performed with initial centroids chosen randomly from the dataset, keeping the solution with the lowest total cost. More sophisticated initialisation techniques exist, such as k -means++ [2], which provide theoretical guarantees of $O(\log k)$ -competitiveness with respect to the optimum; in this project we focus on simple random initialisations, following the approach of [1].

The algorithm converges in a finite number of iterations to a local minimum [1]. The proof follows from the descent structure: both steps guarantee a monotone decrease in the objective function [4], and since the number of possible assignments is finite, the algorithm necessarily terminates. A notable result of [1] is that constrained k -means is less prone to local minima than classical k -means: the capacity constraint forces a more uniform distribution of points among clusters, avoiding degenerate solutions with empty or nearly empty clusters.

1.2.1 Centroid Update Step

As shown in Section 1.1 via Theorems 3.1 and 3.3 [4], when the assignments a_{ij} are fixed, the problem of finding the optimal centroid of cluster j is strictly convex with respect to c_j . The unique global minimum is given by the mean of the assigned points:

$$c_j^* = \frac{\sum_{i=1}^n a_{ij} x_i}{\sum_{i=1}^n a_{ij}} \quad (10)$$

If cluster j is empty ($\sum_i a_{ij} = 0$), the centroid remains unchanged from the previous iteration.

1.2.2 Constrained Assignment Step

With fixed centroids, the constrained assignment step solves the following sub-problem: find the assignment that minimises the total cost while respecting the capacity constraint (3):

$$\min_{a_{ij}} \sum_{i=1}^n \sum_{j=1}^k a_{ij} \|x_i - c_j\|^2 \quad \text{s.t.} \quad \sum_{j=1}^k a_{ij} = 1 \quad \forall i, \quad \sum_{i=1}^n a_{ij} \leq m \quad \forall j, \quad a_{ij} \geq 0 \quad (11)$$

In classical k -means this step is trivial: each point is assigned to its nearest centroid with complexity $O(nk)$. With the constraint m this greedy approach is no longer applicable, since it may violate constraint (3).

Following [1], the constraint structure of (11) is equivalent to a **Minimum Cost Flow (MCF)** problem on a directed bipartite graph $G = (N, A)$, constructed as follows:

- each point x_i is a **source node** with supply $b_{x_i} = +1$;
- each centroid c_j is a **sink node** with demand $b_{c_j} = -m$;
- the **arc cost** (x_i, c_j) is $\|x_i - c_j\|^2$;
- an **artificial node** a is added with supply $b_a = km - n$ and zero-cost arcs (a, c_j) . The artificial node is necessary to balance the graph: the total supply of the points is n , while the total demand of the centroids is km . Since $m \geq \lceil n/k \rceil$ implies $km \geq n$, the $km - n$ unused slots are covered by the artificial node through $km - n$ fictitious zero-cost assignments, which do not correspond to any real point.

Note that [1] originally formulate a *minimum* capacity constraint ($\sum_i a_{ij} \geq \tau_j$). In our problem the capacity is *maximum* ($\sum_i a_{ij} \leq m$): this is reflected in the demand of the sink node, which changes from $b_{c_j} = -\tau_j$ to $b_{c_j} = -m$. However, the integrality guarantee of Proposition 3.1 of [1] rests on two ingredients: the **total unimodularity** of the constraint matrix of the MCF problem — a property of the structure of the directed bipartite graph, independent of the type of capacity constraint — and the **integrality** of the supply and demand values. Both ingredients remain valid in our case: the graph structure is identical, and $b_{c_j} = -m$ is integer by construction ($m \geq \lceil n/k \rceil$). The citation of [1] is therefore justified. In this sense, the two types of constraint are structurally related: both impose a limit on the number of points assigned to each centroid — one from below, the other from above — and both translate into a constraint on the demand of the sink node in the MCF graph, leaving the mathematical structure of the problem unchanged.

Network flow problems enjoy a fundamental structural property: their constraint matrix is **totally unimodular** [6]. A matrix is totally unimodular if, taking any square sub-matrix of rows and columns and computing its determinant, the result is always 0, +1 or -1 — never any other number. This property guarantees that any LP with that constraint structure has integer optimal solutions, provided the right-hand side values are integers.

In the specific case of sub-problem (11), all supply and demand values are integers ($b_{x_i} = 1, b_{c_j} = m$). As a result, the integrality constraint can be *relaxed*: instead of imposing $a_{ij} \in \{0, 1\}$, it is replaced by $a_{ij} \geq 0$, turning the problem into a simple LP. Normally this relaxation could produce fractional solutions meaningless as assignments (e.g. $a_{ij} = 0.3$). However, thanks to total unimodularity, Proposition 3.1 of [1] guarantees that the optimal solution of the relaxation is already automatically integer: $a_{ij} \in \{0, 1\}$. Therefore no MIP solver is needed — it suffices to solve the continuous relaxation using specialised network flow algorithms, which are significantly faster [1]. Implementation details are

described in Chapter 2.

1.3 Exact Approach: L_2 Clustering as a MIQP (A2)

1.3.1 Mixed-Integer Quadratic Programming (MIQP)

Mixed-Integer Quadratic Programming (MIQP) is a mathematical optimisation approach involving both integer and continuous decision variables, with a **quadratic** objective function and linear constraints. It is the natural formulation for problems where discrete decisions are combined with quadratic objectives, and is widely used in all fields where optimal decisions under discrete constraints are required.

1.3.2 Characteristics

MIQP models have several fundamental characteristics. They involve **mixed variables**, incorporating both integer (often binary) and continuous variables. All constraints are expressed through **linear relations**. The optimisation objective is represented by a **quadratic function** — unlike MILP where it is linear. In our case, the objective function is convex quadratic in the auxiliary variables $(z_{ij}^\ell)^2$, as detailed in Section 1.3.6, and is solved via branch-and-bound with continuous relaxations at each node [4]. Due to computational complexity, solving MIQP problems is in general NP-hard, necessitating the use of specialised solvers such as Gurobi [7], CPLEX or OSQP.

1.3.3 Use Cases

MIQP is applied across a wide range of real-world optimisation problems. In combinatorial optimisation, it is used for portfolio selection, facility location and production planning with quadratic objectives. In control engineering, it is employed in Model Predictive Control (MPC) with binary decision variables. In machine learning, it finds application in clustering problems such as ours — where binary assignments combine with a quadratic ℓ_2 objective function — as well as in constrained regression and feature selection.

1.3.4 Strengths

MIQP offers several advantages:

- **Guaranteed optimality:** when solved exactly, MIQP provides the global optimal solution.
- **Exact reformulation:** by introducing auxiliary variables z_{ij}^ℓ , the bilinear product $a_{ij} \cdot c_j^\ell$ is represented exactly with linear constraints, without approximation.
- **Robust solvers:** modern solvers such as Gurobi [7] efficiently handle convex MIQPs via branch-and-bound, cutting planes and advanced presolve techniques.

1.3.5 Weaknesses

Despite its strengths, MIQP has several limitations:

- **Scalability:** solving MIQP on large instances is computationally expensive and can become in-

tractable. The presence of binary variables $a_{ij} \in \{0, 1\}$ makes the problem NP-hard [3].

- **Solution time:** for moderately sized instances, the computation time grows rapidly with n and k , making A2 impractical for large datasets.
- **Solver dependence:** performance depends strongly on the chosen solver and its ability to handle convex MIQP formulations with binary variables.

1.3.6 Decision Variables and Formulation

Given: a set of n points $x_1, x_2, \dots, x_n \in \mathbb{R}^d$ and an integer k representing the number of clusters. The constrained clustering problem under the ℓ_2 norm is formulated as a Mixed-Integer Quadratic Program (MIQP) with linear constraints and a convex quadratic objective function, minimising the total sum of squared Euclidean distances between the points and the k centroids.

Sets and Indices

- $i \in \{1, 2, \dots, n\}$: index over dataset points.
- $j \in \{1, 2, \dots, k\}$: index over clusters.
- $\ell \in \{1, 2, \dots, d\}$: index over dimensions.

Decision Variables

- $c_j^\ell \in [L^\ell, U^\ell]$: ℓ -th coordinate of centroid j (continuous variable).
- $a_{ij} \in \{0, 1\}$: binary assignment variable — equals 1 if point i is assigned to cluster j , 0 otherwise.
- $z_{ij}^\ell \in [\min(0, L^\ell), \max(0, U^\ell)]$: continuous auxiliary variable representing the product $a_{ij} \cdot c_j^\ell$.

Parameters

- $x_i \in \mathbb{R}^d$: coordinates of point i .
- n : number of points.
- k : number of clusters.
- d : dimensionality of the points.
- $m \geq \lceil n/k \rceil$: maximum capacity of each centroid.
- $L^\ell = \min_i x_i^\ell, U^\ell = \max_i x_i^\ell$: lower and upper bounds for dimension ℓ , computed from the data.

Need for a Reformulation The natural objective function of problem (P) is:

$$\min \sum_{i=1}^n \sum_{j=1}^k a_{ij} \|x_i - c_j\|_2^2 \quad (12)$$

Expanding the inner term:

$$a_{ij} \|x_i - c_j\|_2^2 = a_{ij} \|x_i\|_2^2 - 2a_{ij} x_i \cdot c_j + a_{ij} \|c_j\|_2^2 \quad (13)$$

The term $a_{ij} \|c_j\|_2^2 = a_{ij} \sum_{\ell} (c_j^\ell)^2$ is the product of a binary variable and a quadratic — total degree 3. A MIQP solver requires an objective function of degree at most 2, so this expression cannot be used directly.

Auxiliary Variables and Reformulation The problem arises from products of the form $a_{ij} \cdot c_j^\ell$, where $a_{ij} \in \{0, 1\}$ is binary and $c_j^\ell \in [L^\ell, U^\ell]$ is continuous. Since a_{ij} takes only two values, the product can take only two values:

$$a_{ij} \cdot c_j^\ell = \begin{cases} 0 & \text{if } a_{ij} = 0 \\ c_j^\ell & \text{if } a_{ij} = 1 \end{cases} \quad (14)$$

For each pair (i, j) and each dimension ℓ , we introduce the auxiliary variable z_{ij}^ℓ representing exactly this product. Since $c_j^\ell \in [L^\ell, U^\ell]$, the logical relations above translate into the following linear constraints [4]:

$$L^\ell \cdot a_{ij} \leq z_{ij}^\ell \leq U^\ell \cdot a_{ij} \quad (15)$$

$$c_j^\ell - U^\ell(1 - a_{ij}) \leq z_{ij}^\ell \leq c_j^\ell - L^\ell(1 - a_{ij}) \quad (16)$$

Constraint (15) ensures that $z_{ij}^\ell = 0$ when $a_{ij} = 0$: the point is not assigned to cluster j , so the centroid contribution is zero. Constraint (16) ensures that $z_{ij}^\ell = c_j^\ell$ when $a_{ij} = 1$: the point is assigned to cluster j , so the auxiliary variable takes the value of the centroid. Together, the two constraints force exactly $z_{ij}^\ell = a_{ij} \cdot c_j^\ell$ without introducing any nonlinear terms.

Objective Function Since $a_{ij} \in \{0, 1\}$, we have $a_{ij} = a_{ij}^2$, and therefore the degree-3 term simplifies as follows:

$$a_{ij}(c_j^\ell)^2 = a_{ij}^2(c_j^\ell)^2 = (a_{ij} \cdot c_j^\ell)^2 = (z_{ij}^\ell)^2 \quad (17)$$

Substituting $a_{ij} \cdot c_j^\ell = z_{ij}^\ell$ (and hence $a_{ij}(c_j^\ell)^2 = (z_{ij}^\ell)^2$) into the expansion, the problematic term becomes:

$$a_{ij}\|x_i - c_j\|_2^2 = a_{ij}\|x_i\|_2^2 - 2 \sum_{\ell=1}^d x_i^\ell z_{ij}^\ell + \sum_{\ell=1}^d (z_{ij}^\ell)^2 \quad (18)$$

The term $a_{ij}\|x_i\|_2^2$ is linear in a_{ij} since $\|x_i\|_2^2$ is a constant. Summing over all points and clusters, the objective function becomes:

$$\min \sum_{i=1}^n \sum_{j=1}^k \left(a_{ij}\|x_i\|_2^2 + \sum_{\ell=1}^d [(z_{ij}^\ell)^2 - 2x_i^\ell z_{ij}^\ell] \right) \quad (19)$$

The term $(z_{ij}^\ell)^2$ is quadratic and *convex* in the auxiliary variable z_{ij}^ℓ — total degree 2, acceptable by a MIQP solver. The objective function (19) is therefore a **convex MIQP**: all nonlinear terms are convex quadratics in the continuous variables, and no bilinear terms appear. This convex structure allows Gurobi to solve the problem with standard branch-and-bound without requiring any special non-convex solver settings.

Constraints In addition to the constraints already defined in Section 1.1 — unique assignment (2) and maximum capacity (3) — the MIQP formulation adds:

- Centroid domain (redundant constraint: since the optimal centroids are weighted averages of the assigned points, they necessarily belong to the hyper-rectangle $\prod_\ell [L^\ell, U^\ell]$; the constraint is retained to improve Gurobi's presolve):

$$L^\ell \leq c_j^\ell \leq U^\ell, \quad \forall j, \ell \quad (20)$$

- Linearisation constraints (15)–(16) $\forall i, j, \ell$

- Integrality of assignments:

$$a_{ij} \in \{0, 1\}, \quad \forall i, j \quad (21)$$

- Domain of auxiliary variables:

$$z_{ij}^\ell \in [\min(0, L^\ell), \max(0, U^\ell)], \quad \forall i, j, \ell \quad (22)$$

This MIQP formulation guarantees that the optimal centroids are determined by minimising the total sum of squared Euclidean distances, subject to the assignment (2) and capacity (3) constraints. The high computational cost remains a significant limitation, since the presence of binary variables $a_{ij} \in \{0, 1\}$ makes the problem NP-hard [3] and the solution time grows rapidly with n and k . The exact formulation is used primarily to obtain a certified evaluation of the global optimum on sufficiently small instances, in order to assess the quality of the solutions produced by A1 — a purpose for which the high computational cost is acceptable.

The MIQP problem is solved using Gurobi [7], a commercial solver with a free academic licence. Implementation details are described in Chapter 2.

2 Optimization Algorithms

This section describes in detail the two algorithms proposed to solve the constrained clustering problem under the ℓ_2 norm. Section 2.1 presents the heuristic approach A1, based on a modified version of k -means with a constrained assignment step solved via Minimum Cost Flow, and describes its operation in detail. Section 2.3 presents the exact approach A2, based on the MIQP formulation solved with Gurobi [7].

2.1 Heuristic Approach (A1)

The heuristic approach to the constrained clustering problem adapts the classical k -means algorithm [8] by introducing the capacity constraint m : no centroid may receive more than $m \geq \lceil n/k \rceil$ points. The iterative structure alternates a constrained assignment step and a centroid update step, both modified with respect to classical k -means to account for the ℓ_2 norm and the capacity constraint.

A fundamental challenge in clustering is sensitivity to the initial choice of centroids, which can lead to suboptimal solutions [8]. To address this, A1 employs r independent random initialisations, retaining the solution with the lowest total cost. As shown by Bradley et al. [1], the descent structure of both steps guarantees that the objective function never increases, and since the number of feasible assignments is finite, the algorithm necessarily converges to a local minimum [8, 1].

2.1.1 Algorithm 1: Modified k -means with Capacity Constraint

Algorithm 1 Modified k -means with Capacity Constraint (A1)

Require: Dataset $X = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$, number of clusters k , capacity m , number of initialisations r , threshold ϵ

Ensure: C^* , labels*, d_{best}

```

1:  $d_{\text{best}} \leftarrow \infty$ 
2: for  $i \leftarrow 1$  to  $r$  do
3:   Select  $k$  centroids randomly from  $X \rightarrow C$ 
4:   repeat
5:      $C_{\text{old}} \leftarrow C$ 
6:     // Step 1: Constrained Assignment (MCF)
7:     Build  $G=(N, A)$ : source nodes  $x_i$  (supply= 1), nodes  $c_j$  (demand  $m$ ), costs  $\|x_i - c_j\|_2^2$ ,
      artificial node with supply  $km - n$ , arcs  $(a, c_j)$ 
8:     labels  $\leftarrow \text{SolveMinCostFlow}(G)$  ▷ [1, 9]
9:     // Step 2: Centroid Update
10:    for each  $c_j \in C$  do
11:       $c_j \leftarrow \frac{\sum_{i: \text{labels}[i]=j} x_i}{|\{i : \text{labels}[i] = j\}|}$  (if cluster empty:  $c_j \leftarrow C_{\text{old},j}$ )
12:    end for
13:    until  $\max_j \|c_j - C_{\text{old},j}\|_2 < \epsilon$ 
14:     $d_{\text{total}} \leftarrow \sum_{i=1}^n \sum_{j=1}^k a_{ij} \|x_i - c_j\|_2^2$ 
15:    if  $d_{\text{total}} < d_{\text{best}}$  then
16:       $d_{\text{best}} \leftarrow d_{\text{total}}$ ,  $C^* \leftarrow C$ , labels*  $\leftarrow$  labels
17:    end if
18:  end for
19: return ( $C^*$ , labels*,  $d_{\text{best}}$ )

```

2.2 Explanation of Algorithm 1

Algorithm A1 is an extension of classical k -means [8] that solves the constrained clustering problem under the ℓ_2 norm, adapting the assignment step to respect the capacity constraint m via Minimum Cost Flow [1]. The use of r independent random initialisations reduces sensitivity to the initial choice of centroids.

2.2.1 Initialization

r independent runs are performed, each with a different set of k initial centroids chosen randomly from the dataset X . For each run, a vector labels $\in \{1, \dots, k\}^n$ of length n is initialised, where labels[i] indicates the cluster j to which point x_i is assigned. Initially this vector is empty.

2.2.2 Constrained Assignment Step

At the beginning of each iteration, the algorithm saves the current centroids in C_{old} before modifying them: this is necessary to compute the centroid displacement at the end of the iteration and verify the

convergence criterion. The assignment step solves the sub-problem (11) formulated as Minimum Cost Flow on a directed bipartite graph $G = (N, A)$, following [1]:

- each point x_i is a **source node** with supply $b_{x_i} = +1$;
- each centroid c_j is a **sink node** with demand $b_{c_j} = -m$;
- the **arc cost** (x_i, c_j) is $\|x_i - c_j\|_2^2$;
- an **artificial node** a is added with supply $b_a = km - n$ and zero-cost arcs (a, c_j) .

The artificial node is necessary to balance the graph. The total supply of the points is n , while the total demand of the centroids is km . Since $m \geq \lceil n/k \rceil$ implies $km \geq n$, the demand always exceeds the real supply: the $km - n$ unused slots are covered by the artificial node through $km - n$ fictitious zero-cost assignments, which do not correspond to any real point. The total balance is therefore:

$$\underbrace{n}_{\text{points}} + \underbrace{(km - n)}_{\text{artificial}} - \underbrace{km}_{\text{centroids}} = 0$$

The MCF is solved using OR-Tools [9], a Google Python library for graph optimisation, which implements the cost-scaling push-relabel algorithm by Goldberg and Tarjan [10]. This algorithm is significantly faster than a generic LP solver thanks to the specialised structure of the flow problem. Due to the total unimodularity property of the constraint matrix of network flow problems [6], Proposition 3.1 of [1] guarantees that the optimal solution is automatically integer $a_{ij} \in \{0, 1\}$, without requiring a MIP solver.

Note that [1] originally formulate a **minimum** capacity constraint $\sum_i a_{ij} \geq \tau_j$: each centroid must receive *at least* τ_j points. In our problem the capacity is **maximum** $\sum_i a_{ij} \leq m$: each centroid may receive *at most* m points. This is reflected in the demand of the sink node, which changes from $b_{c_j} = -\tau_j$ to $b_{c_j} = -m$. The MCF graph structure and all theoretical guarantees remain identical.

2.2.3 Centroid Update Step

Once the assignments are complete, the centroids are updated by computing the arithmetic mean of the points assigned to each cluster. As shown in Section 1.1 via Theorems 3.1 and 3.3 [4], the objective function is strictly convex in the centroids with fixed assignments, with a positive definite Hessian everywhere: consequently, the mean is the unique global minimiser of the sum of squared ℓ_2 distances with fixed centroids.

If cluster j is empty ($\sum_i a_{ij} = 0$), no point is assigned to c_j and its contribution to the objective function is zero regardless of its position. The centroid therefore remains unchanged from the previous iteration, ensuring that the update step never increases the objective function.

After convergence of the **repeat** loop, once all centroids have stopped moving significantly, the total cost of the solution is computed:

$$d_{\text{total}} = \sum_{i=1}^n \sum_{j=1}^k a_{ij} \|x_i - c_j\|_2^2 \quad (23)$$

The cost is computed only after convergence because only at that point is the solution for this initialisation stable and comparable with the others. If $d_{\text{total}} < d_{\text{best}}$, both C^* — the optimal centroids, describing the cluster positions — and labels* — the optimal assignments, indicating which cluster each point belongs to — are updated. Both pieces of information are necessary: C^* alone does not allow us to know

which points belong to which cluster, while labels* alone does not provide the centroid positions. The algorithm always iterates through all r runs.

2.2.4 Convergence and Final Selection

The iterative process continues until the maximum displacement of any centroid falls below the threshold ϵ :

$$\max_j \|c_j - C_{\text{old},j}\|_2 < \epsilon \quad (24)$$

The **maximum** rather than the mean of the displacements is used because the mean could be small even when a single centroid is still moving significantly. The maximum instead guarantees that *all* centroids have stopped moving before convergence is declared.

Since the MCF step is optimal with fixed centroids and the mean update step is optimal with fixed assignments [4], both steps guarantee a monotone decrease in the objective function [8, 1]. Combined with the fact that the number of assignments is finite, the algorithm necessarily terminates at a local minimum [1, 8].

Once all r runs are complete, the final solution corresponds to the centroids and assignments that produced the lowest total cost across all initialisations.

2.2.5 Computational Complexity

The computational complexity of A1 depends on n , k , d and r .

Assignment step (MCF). The graph has $n + k + 1$ nodes and $nk + k$ arcs, where the nodes are n sources (the points x_i), k sinks (the centroids c_j) and 1 artificial node a ; the arcs are nk arcs (x_i, c_j) and k arcs (a, c_j) . OR-Tools uses the cost-scaling push-relabel algorithm [10] with complexity $O(N^2 M \log(N \cdot C))$, where N is the number of nodes, M the number of arcs and C the maximum arc cost [9]. In our case $N = O(n + k)$, $M = O(nk)$ and $C \leq d \cdot \max_\ell (U^\ell - L^\ell)^2$ (the squared Euclidean distance between two points in the hyper-rectangle $\prod_\ell [L^\ell, U^\ell]$ is at most $\sum_\ell (U^\ell - L^\ell)^2 \leq d \cdot \max_\ell (U^\ell - L^\ell)^2$), so the complexity of the assignment step is:

$$O((n + k)^2 \cdot nk \cdot \log((n + k) d)) \quad (25)$$

For $k \ll n$ this simplifies to $O(n^3 k \log(nd))$.

Update step. Computing the mean for each cluster requires $O(nd)$: in the worst case all n points belong to the same cluster and are summed over d dimensions.

Total complexity. Assuming I iterations per run, where I is typically small and independent of n in practice [8], the total complexity over r runs is:

$$O(r \cdot I \cdot n^3 k \log(nd)) \quad (26)$$

This complexity is higher than that of classical k -means $O(r \cdot I \cdot nk d)$ [8] due to the MCF step. However, the MCF step is necessary to guarantee the optimality of the assignment while respecting the capacity constraint m — a computational cost that classical k -means does not incur since it has no capacity constraints.

2.3 Exact Approach (A2)

The exact approach to the constrained clustering problem applies the Gurobi solver [7] to the MIQP formulation described in Section 1.3.

2.3.1 Algorithm 2: MIQP for Constrained Clustering

Algorithm 2 MIQP for Constrained Clustering (A2)

Require: Dataset $X = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$, number of clusters k , capacity m

Ensure: Optimal centroids C^* , assignments labels*, optimal cost d^*

```

1: Initialise Gurobi model [7]
2:  $a_{ij} \in \{0, 1\}$   $\forall i \in \{1, \dots, n\}, j \in \{1, \dots, k\}$ 
3:  $c_j^\ell \in [L^\ell, U^\ell]$   $\forall j \in \{1, \dots, k\}, \ell \in \{1, \dots, d\}$ 
4:  $z_{ij}^\ell \in [\min(0, L^\ell), \max(0, U^\ell)]$   $\forall i, j, \ell$ 
5: for each  $i$  do
6:    $\sum_{j=1}^k a_{ij} = 1$   $\triangleright$  unique assignment (2)
7: end for
8: for each  $j$  do
9:    $\sum_{i=1}^n a_{ij} \leq m$   $\triangleright$  maximum capacity (3)
10: end for
11: for each  $i, j, \ell$  do
12:    $L^\ell \cdot a_{ij} \leq z_{ij}^\ell \leq U^\ell \cdot a_{ij}$   $\triangleright$  V1 [4]
13:    $c_j^\ell - U^\ell(1 - a_{ij}) \leq z_{ij}^\ell \leq c_j^\ell - L^\ell(1 - a_{ij})$   $\triangleright$  V2 [4]
14: end for
15:  $\text{obj} \leftarrow \sum_{i=1}^n \sum_{j=1}^k \left( a_{ij} \|x_i\|^2 + \sum_{\ell=1}^d [(z_{ij}^\ell)^2 - 2x_i^\ell z_{ij}^\ell] \right)$   $\triangleright$  eq. (19)
16: Minimise obj using Gurobi [7]
17:  $C^* \leftarrow$  optimal values of  $c_j^\ell$ 
18: labels*  $\leftarrow j$  such that  $a_{ij} = 1$ , for each  $i$ 
19:  $d^* \leftarrow$  optimal value of the objective function
    return ( $C^*$ , labels*,  $d^*$ )

```

2.4 Explanation of Algorithm 2

Algorithm A2 is a direct application of the MIQP formulation of Section 1.3 to the Gurobi solver [7]. Unlike A1, there is no ad-hoc iterative structure: the algorithm simply declares the variables, imposes the constraints, defines the objective function, and delegates the entire resolution to the solver. Its structure is divided into three phases: model construction, resolution via branch-and-bound, and solution extraction.

2.4.1 Model Construction

Lines 2–4 declare the three families of decision variables. Declaring a variable in Gurobi does not mean assigning it a value: it means communicating to the solver that the variable is an unknown to

be optimised, with the specified domain. The binary variables $a_{ij} \in \{0, 1\}$ encode the assignments: $a_{ij} = 1$ if point x_i is assigned to cluster j , 0 otherwise. The continuous variables $c_j^\ell \in [L^\ell, U^\ell]$ represent the ℓ -th coordinate of centroid j . The auxiliary variables $z_{ij}^\ell \in [\min(0, L^\ell), \max(0, U^\ell)]$ — for point i , cluster j and dimension ℓ — are introduced to eliminate degree-3 terms from the objective function: as shown in Section 1.3, the product $a_{ij} \cdot c_j^\ell$ — where a_{ij} is binary and c_j^ℓ is continuous — can take only two values: zero if $a_{ij} = 0$, or c_j^ℓ if $a_{ij} = 1$. Constraints V1 and V2 at lines 11–15 encode this logic exactly via linear constraints [4]: Gurobi does not know the relationship between z_{ij}^ℓ and a_{ij} a priori, but *deduces* it during optimisation from the fact that the linear constraints, combined with the binary nature of a_{ij} , mathematically force the correct behaviour.

Lines 5–10 impose the structural constraints of problem (P): the unique assignment constraint (2) ensures that every point belongs to exactly one cluster; the capacity constraint (3) ensures that no centroid receives more than m points.

The objective function at line 16 is formulation (19) from Section 1.3: convex quadratic in the auxiliary variables $(z_{ij}^\ell)^2$, total degree 2, with all linear constraints. It therefore constitutes a valid **convex** MIQP formulation that can be passed directly to Gurobi.

2.4.2 Resolution with Gurobi and Branch-and-Bound

Line 17 delegates the entire resolution to Gurobi [7], which internally employs the branch-and-bound method [4]. The algorithm relaxes the integrality constraint $a_{ij} \in \{0, 1\}$ to obtain a **lower bound** on the optimal cost; it then refines the solution by branching on the binary variables, generating a tree of sub-problems. Sub-problems whose lower bound exceeds the best current integer solution (**upper bound**) are eliminated without exploration (*pruning*). The algorithm terminates when lower and upper bounds coincide, guaranteeing that the returned solution is the global optimum d^* . This optimality guarantee is the fundamental difference with respect to A1, which finds the best d_{best} among the r initialisations without any certainty on the gap from the optimum. The worst-case complexity is exponential in the number of binary variables $n \times k$, making A2 feasible only on limited-size instances.

2.4.3 Solution Extraction and Optimality Gap

Lines 18–20 extract the solution from the solved model. The optimal centroids C^* are obtained directly from the values of c_j^ℓ ; the assignments labels* are obtained from the binary variables a_{ij} : for each point i , the assigned cluster is the j such that $a_{ij} = 1$. The optimal value is defined as $d^* = d_{A2}$.

The GAP is not part of algorithm A2 itself: it is computed a posteriori, after running both algorithms on the same instance, as:

$$\text{GAP} = \frac{d_{A1} - d_{A2}}{d_{A2}} \times 100\% \quad (27)$$

where d_{A1} is the cost of the heuristic solution and $d_{A2} = d^*$ is the global optimum certified by A2. A small GAP indicates that A1 produces solutions of quality close to the global optimum, despite its heuristic nature.

2.4.4 Computational Complexity

The model has $n \times k$ binary variables, $k \times d$ continuous variables for the centroids, and $n \times k \times d$ auxiliary variables. The presence of binary variables makes the problem NP-hard [3]: in the worst case, branch-and-bound explores a tree of exponential size in $n \times k$. For large instances where A2 does not terminate in reasonable time, a time limit can be set in Gurobi: the solver returns the best integer solution found so far together with the current optimality gap (difference between lower and upper bound), signalling that the solution is not necessarily optimal. A2 is therefore in principle feasible only for limited-size instances. The experimental analysis of Chapter 3 will empirically verify up to which values of n and k A2 can terminate within acceptable times, and on those instances will compute the GAP to assess the quality of the solutions produced by A1.

3 Experiments

In this chapter we evaluate the empirical behaviour of A1 and A2 on synthetic instances. Section 3.1 describes the experimental environment, the data used and the structure of the experiments. Section 3.2 reports the experiments on A1 convergence, A2 scalability and the quality of heuristic solutions with respect to the global optimum. Section 3.3 analyses the effect of the capacity constraint m on the cost and balance of the clusters. Section 3.4 discusses the overall results.

3.1 Experimental Setup and Dataset

The experiments are implemented in Python 3 and organised into three distinct modules — `Dataset.py`, `algorithm1.py` and `algorithm2.py` — coordinated by a single execution notebook. `algorithm1.py` implements A1 with the MCF constrained assignment step via OR-Tools [9] and the convergence criterion described in Section 2.1; `algorithm2.py` implements A2 by constructing the convex MIQP model described in Section 1.3.6 and delegating the resolution to Gurobi [7], which solves it via standard branch-and-bound without requiring any non-convex solver settings, thanks to the quadratic convex objective obtained through the auxiliary variables z_{ij}^ℓ (equation (19)).

Dataset. All experiments use synthetic data generated by sampling n points from a mixture of k isotropic Gaussians with standard deviation σ , whose centres $\mu_j \in \mathbb{R}^2$ are drawn independently from $\mathcal{N}(0, I_2)$ and n_j points are sampled from $\mathcal{N}(\mu_j, \sigma^2 I_2)$. The bounds $L^\ell = \min_i x_i^\ell$ and $U^\ell = \max_i x_i^\ell$ defining the hyper-rectangle $\prod_\ell [L^\ell, U^\ell]$ are computed from the generated data, consistently with Section 1.1. The choice of synthetic Gaussian data is motivated by two reasons: the L_2 norm is the natural metric for isotropic Gaussian distributions (i.e. with covariance matrix $\sigma^2 I$, uniform in all directions), and parametric generation allows exact control over the cluster structure and its interaction with the capacity constraint.

For experiments involving only A1 (E1, E2, Section 3.3) we use three fixed configurations in $\mathbb{R}^2$ with $n = 90$, $k = 3$ and $m = \lceil 90/3 \rceil = 30$, designed to cover three qualitatively distinct regimes of the capacity constraint:

Config	Cluster sizes	σ	Constraint active?	Redistributed points
Easy	30/30/30	0.8	No	0
Medium	40/30/20	1.4	Yes	10
Hard	50/25/15	2.0	Yes	20

The constraint is *active* when $\max_j n_j > m$, i.e. when A1 is forced to redistribute points with respect to the natural data structure. For experiments involving A2 (E3–E5, `E_gap_m`) we instead generate independent random instances with centres resampled for each instance and $\sigma = 1$. Figure 1 shows the three configurations.

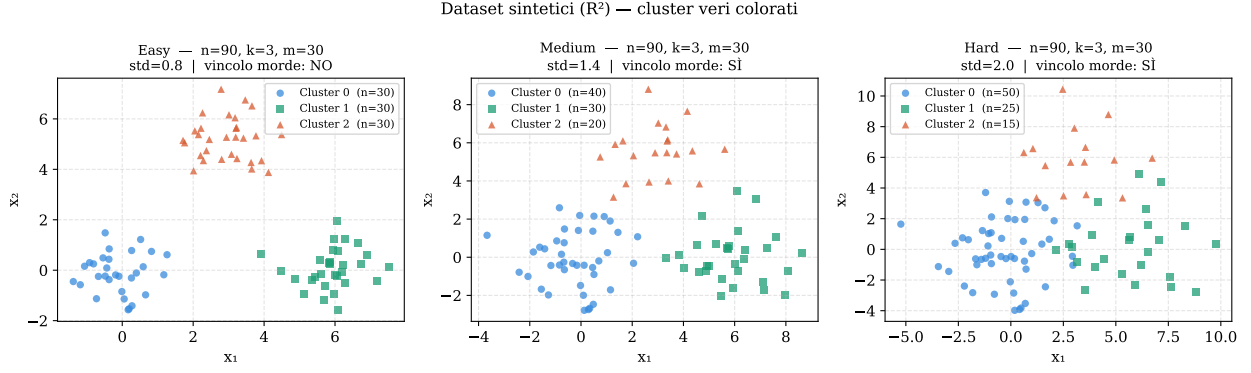


Figure 1: The three synthetic datasets: easy, medium and hard.

Overview of experiments. We structured the experimental campaign into six main experiments, each with a precise purpose:

- **E1 — Monotone decrease:** empirical verification of the theoretical descent property of A1 (Section 2.1) on the three fixed datasets easy, medium and hard.
- **E2 — A1 convergence:** measures how the number of iterations and the execution time of A1 scale as n grows, for $k \in \{2, 3, 4\}$.
- **E3 — A2 scalability:** empirically characterises the exponential growth of Gurobi’s runtime as n grows, estimating the parameters of the exponential model and the maximum value $n_{\max}$ within which A2 is tractable.
- **E4 — GAP A1 vs A2:** measures the quality of A1’s solutions with respect to the global optimum certified by A2, using a single restart ($r = 1$) to highlight the real GAP distribution.
- **E5 — Required number of restarts:** building on the results of E4, estimates how many restarts r are sufficient to find the global optimum with high probability.
- **E6 + E_gap_m — Effect of m :** studies how varying the capacity constraint m influences the total cost, cluster balance and the quality of A1 with respect to the optimum.

Fixed parameters. The following parameters are kept constant across all experiments:

- $d = 2$: the number of auxiliary variables z_{ij}^ℓ in the MIQP formulation scales as $n \times k \times d$; with $d = 2$ A2 remains tractable for the instances needed for the GAP analysis. With $d = 4$ (as in the Iris dataset) A2 times out even for very small n .
- $m = \lceil n/k \rceil$: minimum admissible capacity, the most stringent choice that makes the constraint active in the greatest number of cases.
- $\varepsilon = 10^{-6}$: convergence threshold for A1 — $\max_j \|c_j - c_j^{\text{old}}\|_2 < \varepsilon$ ensures that all centroids have stabilised.
- $\text{SEED} = 42$: global seed for complete reproducibility of all results.

The number of restarts r and the size n vary between experiments and are specified in the following sections.

3.2 Experiments on Synthetic Data

3.2.1 Monotone Decrease of A1's Objective

The descent structure of A1 guarantees that the objective function $\sum_i \|x_i - c_{\ell(i)}\|^2$ never increases at each iteration (Section 2.1): the MCF step is optimal with fixed centroids and the mean update step is optimal with fixed assignments [4], so neither step can increase the cost. Figure 2 verifies this property empirically: for three independent initialisations on each of the three fixed datasets, all trajectories are monotonically non-increasing, confirming the theoretical guarantee regardless of the data structure. The easy dataset converges in 2–3 iterations; the hard dataset requires up to 4, consistently with the greater overlap that makes each step less decisive.

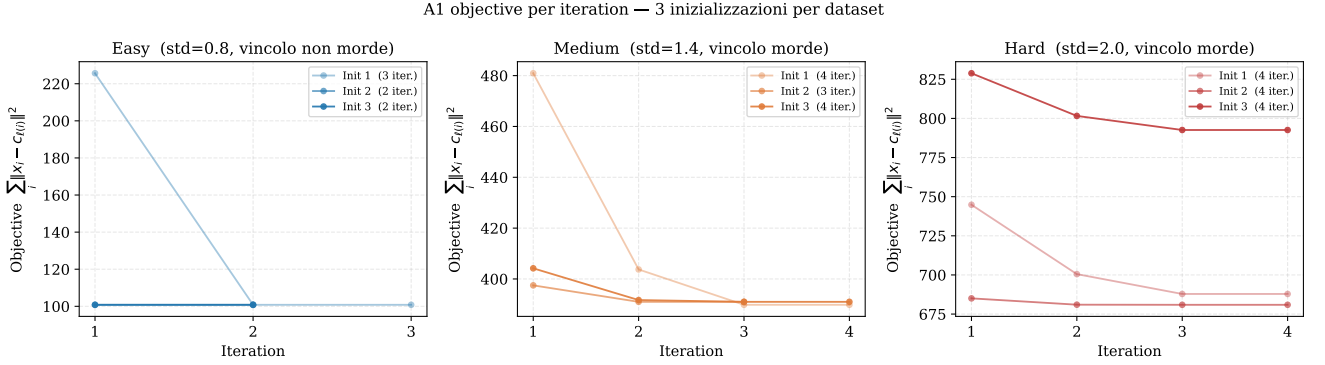


Figure 2: Monotone decrease of A1's objective.

Figure 3 shows the clustering produced by A1 with $r = 10$ restarts on each dataset. In all three cases the centroids (marked with $\star$) lie at the geometric centre of the assigned points, consistently with equation (10), and the capacity constraint $m = 30$ is satisfied exactly: all clusters contain 30 points even in the hard dataset where the constraint is most binding.

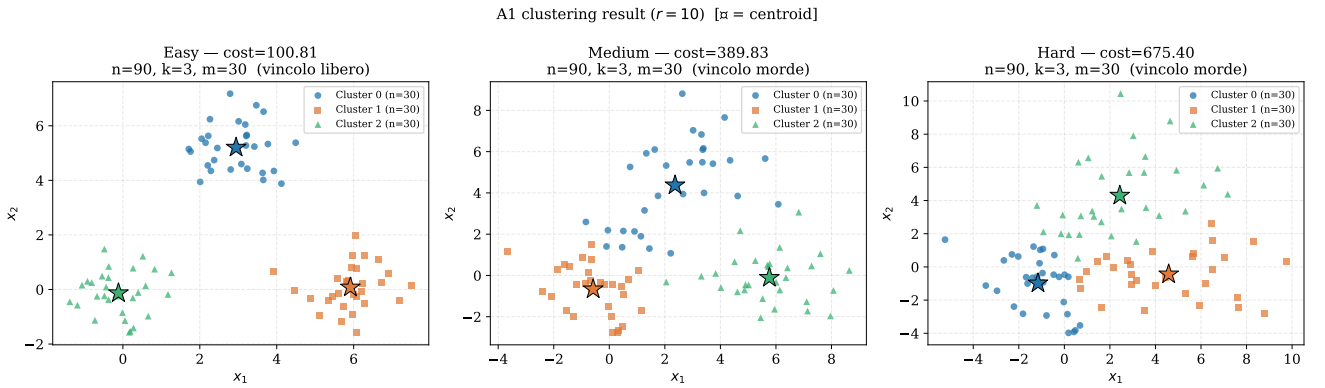


Figure 3: Clustering produced by A1 ($r = 10$).

3.2.2 A1 Convergence: Iterations and Time as a Function of n

To evaluate how the convergence speed of A1 scales with the problem size, we generated 30 random instances for each pair (n, k) with $n \in \{20, 40, \dots, 200\}$ and $k \in \{2, 3, 4\}$, running A1 with $r = 10$ restarts on each. Figure 4 reports the results.

The left panel shows the mean number of iterations to convergence together with ± 1 standard deviation. Consistently with convergence guaranteed in a finite number of iterations [1, 8], the number of iterations

is small in practice: it grows slowly from about 3 for $n = 20$ to about 8 for $n = 200$, with a maximum of 12 observed on the tested instances (this is not a limit imposed by the code, but an empirical result). Larger values of k require on average more iterations, since a greater number of centroids must stabilise simultaneously. The right panel shows the total execution time of A1 ($r = 10$ restarts included): it grows regularly with n , reaching approximately 0.2 seconds for $n = 200$ and $k = 4$. The cost of the MCF step scales as $O(n^3 k \log(nd))$ (Section 2.1), but the small and substantially stable number of iterations means that the overall time grows much more slowly in practice.

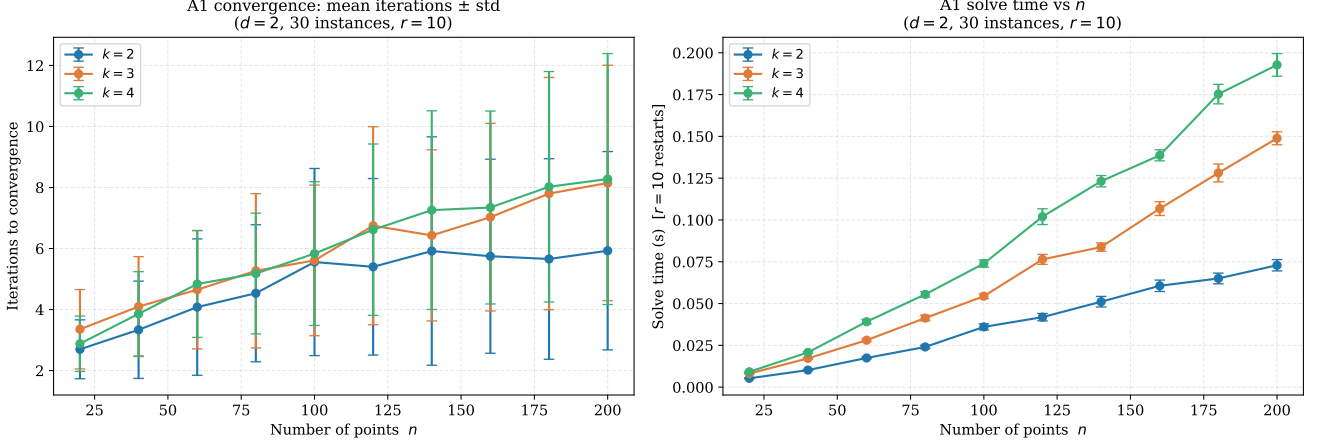


Figure 4: A1 convergence as a function of n for $k \in \{2, 3, 4\}$ ($d = 2, 30$ instances, $r = 10$).

3.2.3 A2 Scalability

A2 solves a convex MIQP via Gurobi [7] (Section 2.3). The convex quadratic objective — obtained through the auxiliary variables z_{ij}^ℓ as detailed in Section 1.3.6 — allows Gurobi to apply standard branch-and-bound with convex relaxations at each node, without requiring any non-convex solver settings. Despite this structural advantage, the problem remains NP-hard [3] because of the binary assignment variables $a_{ij} \in \{0, 1\}$, and the runtime still grows rapidly with the number of points. To characterise this growth empirically, we measured the mean solution time on five random instances for each pair (n, k) , starting from $n = 5$ and incrementing n until the average runtime exceeded the threshold of 60 seconds.

The observed runtimes were fitted using the exponential model

$$\text{time}(n) = c \cdot 10^{\gamma n},$$

which becomes linear in logarithmic scale:

$$\log_{10}(\text{time}) = \gamma n + \log_{10}(c).$$

The parameters were estimated through ordinary least-squares linear regression on the log-transformed runtimes, excluding the instances whose average runtime reached the 60-second threshold. Figure 5 shows the measured runtimes together with the fitted exponential curves on a logarithmic scale; Table 1 reports the estimated parameters.

The parameters have a direct interpretation in the exponential model:

- $\hat{\gamma}$ is the *exponential growth rate*, corresponding to the slope of the fitted line in the log-linear representation. Larger values indicate a faster increase in computational cost as the instance size grows.

- $\log_{10}(\hat{c})$ is the *intercept* of the fitted line, representing the logarithm of the multiplicative constant in the exponential model.
- $n_{\max}$ is the *largest value of n* for which A2 is able to solve the problem within the 60-second time limit. This quantity determines the maximum instance size considered in the subsequent experiments.
- R^2 is the *coefficient of determination*. Values close to one indicate that the exponential model explains most of the variability observed in the logarithm of the solution times.

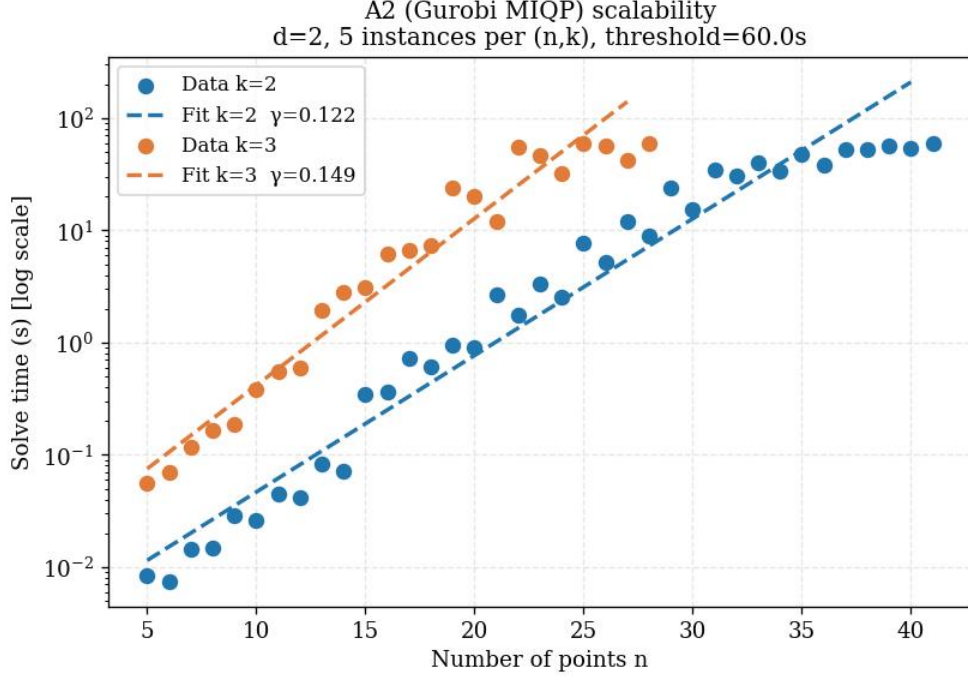


Figure 5: A2 solution time (Gurobi convex MIQP) with exponential fit.

The estimated growth rates are $\hat{\gamma} = 0.1217$ for $k = 2$ and $\hat{\gamma} = 0.1486$ for $k = 3$. The larger value obtained for $k = 3$ indicates that increasing the number of clusters significantly amplifies the computational burden of the branch-and-bound procedure, since each additional cluster introduces n extra binary assignment variables. The resulting exponential growth is consistent with the combinatorial nature of the underlying NP-hard optimisation problem. The threshold of 60 seconds is reached at $n_{\max} = 40$ for $k = 2$ and $n_{\max} = 27$ for $k = 3$, confirming that the exact MIQP formulation is suitable only for relatively small instances. The coefficients of determination are very high ($R^2 = 0.9598$ for $k = 2$ and $R^2 = 0.9522$ for $k = 3$), showing that the exponential model provides an excellent description of the observed runtime growth. As reported in Table 1:

k	$n_{\max}$	$\hat{\gamma}$	$\log_{10}(\hat{c})$	R^2
2	40	0.1217	-2.5483	0.9598
3	27	0.1486	-1.8670	0.9522

Table 1: Parameters of the exponential fit $\log_{10}(\text{time}) = \gamma n + \log_{10}(c)$ for the A2 solution time as a function of n .

3.2.4 GAP between A1 and the Global Optimum

We evaluate the quality of A1’s solutions by comparing them with the global optimum certified by A2. For each instance we run A1 with a *single* random initialisation ($r = 1$) and compute the GAP as defined in equation (27), where $d^* = d_{A2}$ is the exact optimum. Using $r = 1$ measures the real GAP distribution without the benefit of multiple restarts, making the experiment complementary to the analysis in the following section.

We use $n = 12$ for $k = 2$ and $n = 8$ for $k = 3$: values well within the tractable range established in Section 3.2.3, chosen to ensure reliable termination across all 200 independent instances. The results are based on 200 instances for $k = 2$ and 100 for $k = 3$. Table 2 summarises the main statistics; Figure 6 shows the GAP distributions.

The quality of A1’s solutions depends strongly on the instance structure. For $k = 2$, 72% of instances are solved to optimality by a single restart, while for $k = 3$ this percentage drops to 43%. The mean GAP is dominated by a small number of hard instances where A1 converges to a poor local minimum, as evidenced by the long right tail of the distribution. The deterioration with k is expected: as k grows, the number of feasible assignments increases combinatorially, making it exponentially less likely that a random initialisation falls in the basin of attraction of the global optimum.

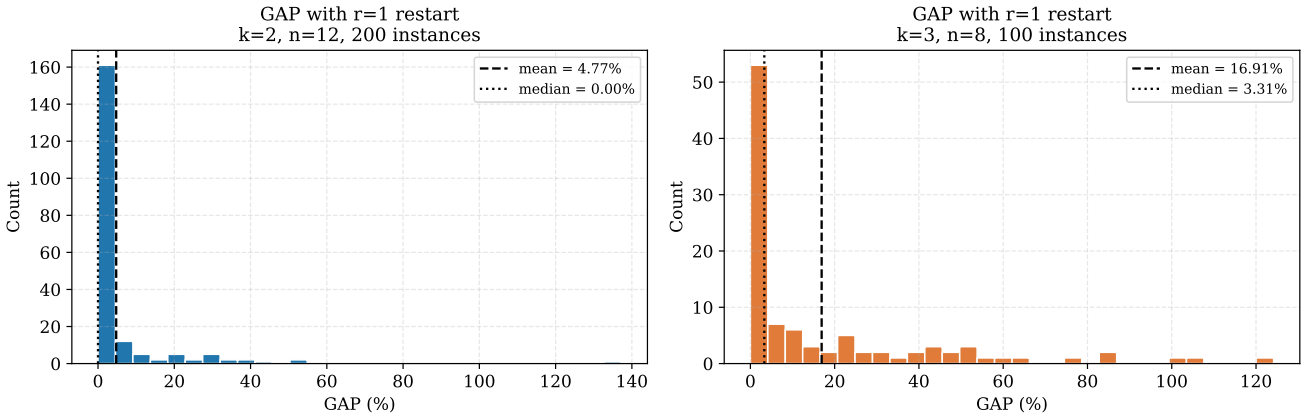


Figure 6: GAP distribution for A1 with $r = 1$ restart.

As can be seen from Table 2, the median GAP is zero for $k = 2$ — half of the instances are already solved to optimality — while for $k = 3$ the median rises to 3.31%, with a maximum tail of 184.86%:

k	n	Instances	Mean GAP	Median GAP	Max GAP	Exact (%)
2	12	200	4.77%	0.00%	137.22%	72.0
3	8	100	16.91%	3.31%	184.86%	43.0

Table 2: GAP statistics (eq. (27)) for A1 with $r = 1$ restart. The column “Exact (%)” indicates the fraction of instances where A1 reaches the exact optimum ($\text{GAP} < 10^{-4}\%$).

3.2.5 Required Number of Restarts

From the per-restart success probability $\bar{s}$ — estimated empirically as described below — we derive the number of restarts r_p needed to find the optimum with probability at least p .

The probability of *not* finding the optimum in a single restart is $(1 - \bar{s})$. With r_p independent restarts,

the probability of failing all of them is $(1 - \bar{s})^{r_p}$. Requiring this probability to be at most $(1 - p)$:

$$(1 - \bar{s})^{r_p} \leq 1 - p \implies r_p \log(1 - \bar{s}) \leq \log(1 - p) \implies r_p \geq \frac{\log(1 - p)}{\log(1 - \bar{s})}$$

where the inequality reverses because $\log(1 - \bar{s}) < 0$. Taking the minimum integer satisfying the constraint:

$$r_p = \left\lceil \frac{\log(1 - p)}{\log(1 - \bar{s})} \right\rceil. \quad (28)$$

$\bar{s}$ is the *mean success probability per single random initialisation* — the frequency with which A1 with $r = 1$ finds the global optimum ($\text{GAP} < 10^{-4}\%$). It is not known a priori: it depends on the difficulty of the instance and the data structure. We estimate it empirically by running A1 with $r = 1$ for $N = 100$ independent times on each instance, counting how many times it finds the optimum certified by A2, and averaging over 100 instances for $k = 2$ and 50 for $k = 3$. We report $\bar{s}$ in Table 3 because it directly determines r_p via equation (28): a higher $\bar{s}$ means fewer restarts are needed.

Figure 7 shows r_p as a function of p .

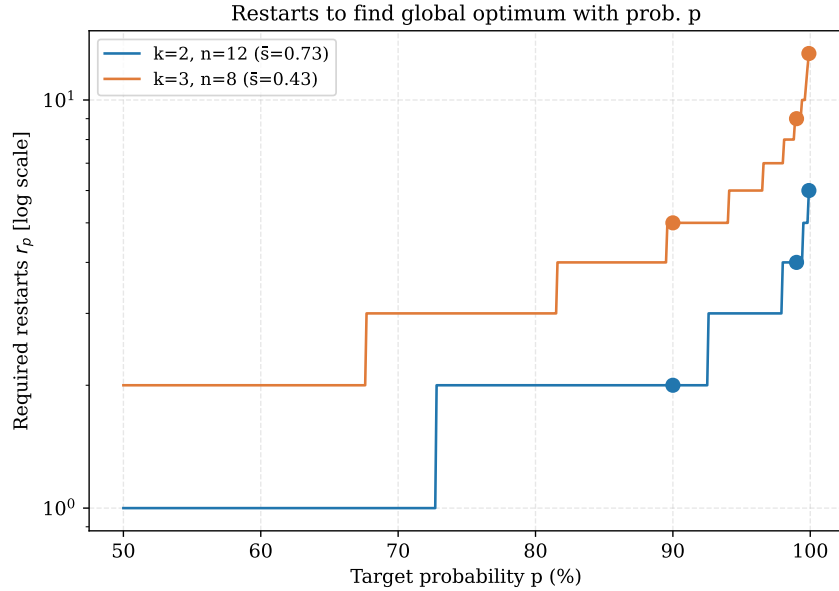


Figure 7: Required number of restarts r_p as a function of p .

The required number of restarts is modest: for $k = 2$, a single restart already suffices for probability up to 73%, and just 4 restarts are enough for 99%; for $k = 3$ the corresponding values are 5 and 9. These results confirm the practical effectiveness of A1 [1]: with a small number of independent initialisations, the algorithm finds the global optimum with high probability. As expected, $k = 3$ requires more restarts because $\bar{s}$ is lower (0.43 vs 0.73). As can be seen from Table 3:

k	n	$\bar{s}$	$r_{90\%}$	$r_{99\%}$	$r_{99.9\%}$
2	12	0.73	2	4	6
3	8	0.43	5	9	13

Table 3: Mean success probability per single initialisation ($\bar{s}$) and required number of restarts (r_p , eq. (28)).

3.3 Effect of the Capacity Constraint m

The parameter m is the defining feature of problem (P) with respect to unconstrained k -means [8]. We study its effect on three quantities: the total clustering cost, cluster balance, and the quality of A1 with respect to the global optimum.

Effect on the GAP (E_gap_m) — original contribution. To assess whether the constraint influences the *quality* of A1 with respect to the global optimum, we computed the GAP between A1 ($r = 10$) and A2 for $k = 2$, $n = 12$, $d = 2$, varying m from $m_{\min} = 6$ to $m_{\max} = 12$ over 30 instances for each value of m . Figure 8 shows the mean cost of A1 and A2 and the mean GAP as a function of m .

The optimal cost (A2) decreases monotonically as m grows from $m_{\min}$: a larger capacity relaxes the feasible region, allowing the solver to reach lower-cost solutions. The cost of A1 ($r = 10$) tracks the optimum closely across all values of m . The GAP remains below approximately 1.5% for $m \geq 9$ and is essentially zero at $m_{\min} = 6$, where the highly constrained feasible region leaves little room for suboptimal local minima. The slight increase of the GAP as m grows is consistent with the theoretical expectation of Bradley et al. [1]: less stringent constraints bring the problem closer to unconstrained k -means, which has more local minima. The effect remains small because with $k = 2$ and $n = 12$ the instances are too small for it to emerge clearly.

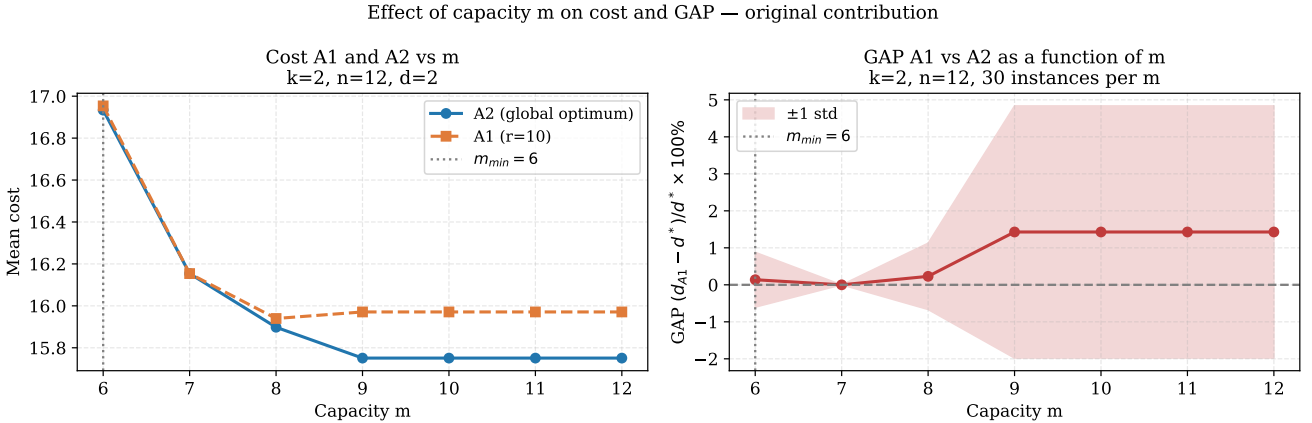


Figure 8: Effect of m on cost and GAP ($k = 2$, $n = 12$, $d = 2$).

Effect on cost and balance (E6). We vary m from $m_{\min} = \lceil n/k \rceil = 30$ to $m_{\max} = n = 90$ on the three fixed datasets with $n = 90$, $k = 3$, averaging over 20 instances for each value of m , and compare A1 against unconstrained scikit-learn k -means as a baseline.

Figure 9 shows the mean total cost as a function of m . For the easy dataset the cost is essentially constant for all values of m : since the natural cluster sizes (30/30/30) already satisfy the constraint for any $m \geq 30$, the constraint is never active. For the medium and hard datasets the cost decreases rapidly as m grows from $m_{\min}$, stabilising at the level of unconstrained k -means. The transition occurs at $m \approx 40$ for medium and at $m \approx 50$ for hard, exactly where the constraint ceases to be active. The cost at $m = m_{\min}$ is 23.6% higher than the unconstrained optimum for medium and 12.6% for hard.

Figure 10 shows the maximum cluster size as a function of m . For easy the maximum remains at 30 regardless of m . For medium it grows from 30 towards 40 as m increases; for hard it grows from 30 towards 45. This illustrates the fundamental trade-off: the capacity constraint (3) reduces cluster heterogeneity at the cost of increasing the objective function (1).

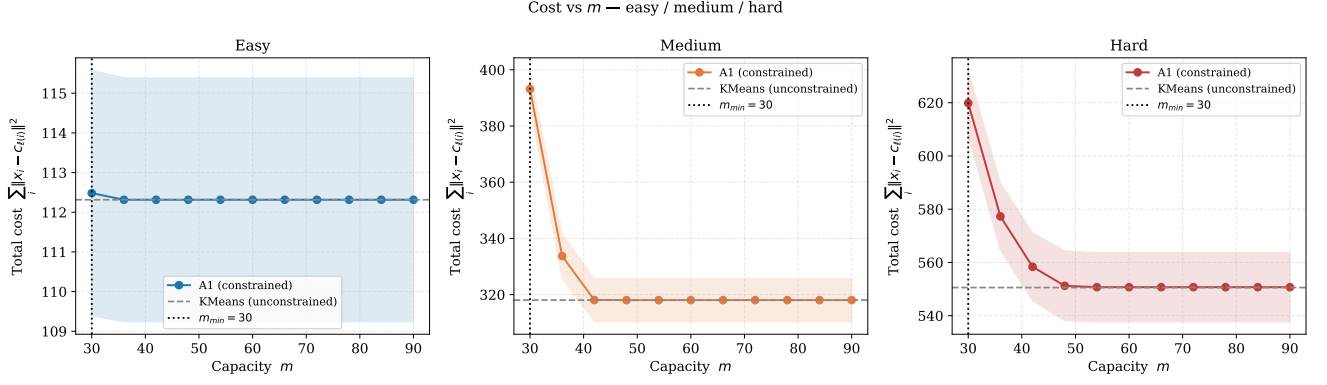


Figure 9: Mean total cost of A1 as a function of m (easy/medium/hard).

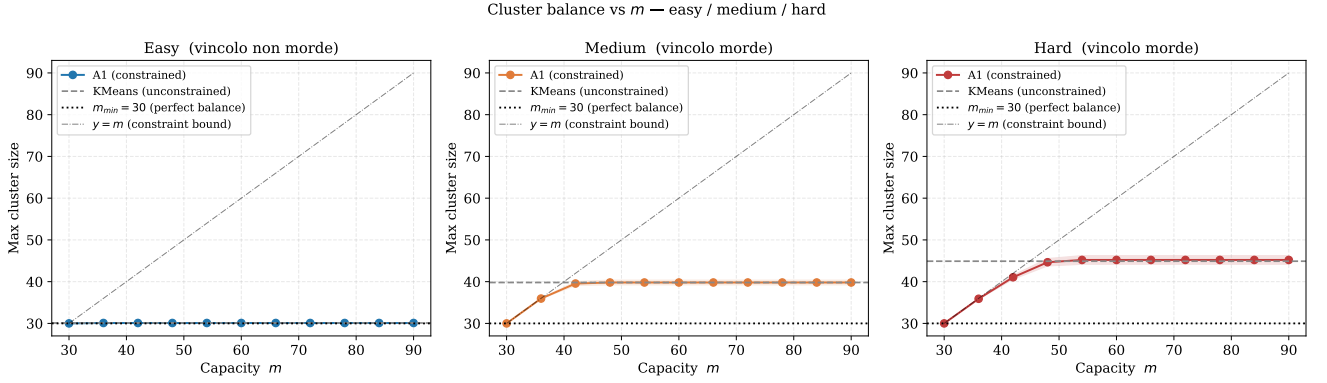


Figure 10: Maximum cluster size as a function of m (easy/medium/hard).

3.4 Discussion

The experimental results paint a coherent picture of the two algorithms and the effect of the capacity constraint.

A1 is efficient and produces high-quality solutions. A1 converges in a small and substantially stable number of iterations as n grows (between 3 and 8 on average), with an execution time that remains below 0.2 seconds even for $n = 200$ and $r = 10$ restarts. The monotone decrease of the objective is verified empirically across all tested configurations, confirming the theoretical guarantee [1]. With a small number of restarts ($r \leq 10$), A1 finds the global optimum with high probability: 72% of instances with $k = 2$, $n = 12$ are solved to optimality by a single random initialisation, and just 4 restarts are sufficient to guarantee 99% probability of success.

A2 certifies the optimum but does not scale. The solution time of A2 grows exponentially with n , with estimated growth rate $\hat{\gamma} = 0.1217$ for $k = 2$ and $\hat{\gamma} = 0.1486$ for $k = 3$. The 60-second threshold is reached at $n_{\max} = 40$ for $k = 2$ and $n_{\max} = 27$ for $k = 3$, making A2 unusable on real-sized instances. The problem remains NP-hard [3] despite the convex MIQP formulation, since the exponential worst-case complexity is determined by the binary assignment variables $a_{ij} \in \{0, 1\}$ rather than by the structure of the continuous objective. The role of A2 is exclusively to certify the global optimum on small instances in order to evaluate the quality of A1 (Section 2.3).

The capacity constraint introduces a cost–balance trade-off. When constraint (3) is active, the optimal cost increases with respect to unconstrained k -means [8], and the magnitude of the increase

depends on the natural cluster imbalance: negligible for easy, moderate for medium (+23.6%) and significant for hard (+12.6%). In return, the constraint guarantees that each cluster contains at most m points, improving balance in a controlled manner. As observed by Bradley et al. [1], constrained k -means is also less prone to local minima than classical k -means: the capacity constraint forces a more uniform distribution of points among clusters, avoiding degenerate solutions.

4 Conclusions

In this project we addressed the constrained clustering problem under the L_2 norm (P), which extends classical k -means by introducing a capacity constraint $m \geq \lceil n/k \rceil$ per centroid. We proposed two approaches: a heuristic (A1) based on modified k -means with a Minimum Cost Flow assignment step, and an exact method (A2) based on a convex MIQP formulation solved with Gurobi.

Theoretical contributions. On the theoretical side, we showed that: the centroid update of A1 always finds the exact optimum, by strict convexity of the objective with fixed assignments (Theorems 3.1 and 3.3 [4]); the constrained assignment step reduces to a Minimum Cost Flow problem whose totally unimodular structure guarantees integer solutions [6, 1], eliminating any MIP solver in the heuristic step; and A2 admits an exact convex MIQP reformulation via auxiliary variables z_{ij}^ℓ that eliminate all degree-3 terms — exploiting the fact that $a_{ij} \in \{0, 1\}$ implies $a_{ij}(c_j^\ell)^2 = (z_{ij}^\ell)^2$ — yielding a degree-2 convex objective passable directly to Gurobi [4].

Experimental results. A1 proves to be an efficient, high-quality heuristic: it converges in a small number of iterations (between 3 and 8 on average, independently of n) with an execution time that remains below 0.2 seconds even for $n = 200$ and $r = 10$. The monotone decrease of the objective is verified empirically across all tested configurations. With $r = 4$ restarts, the probability of finding the global optimum exceeds 99% for $k = 2$; for $k = 3$, $r = 9$ restarts suffice.

A2 certifies the global optimum but scales exponentially with n : the solution time follows the model $\text{time}(n) = c \cdot 10^{\gamma n}$ with $\hat{\gamma} = 0.1217$ for $k = 2$ and $\hat{\gamma} = 0.1486$ for $k = 3$. The 60-second threshold is reached at $n_{\max} = 40$ for $k = 2$ and $n_{\max} = 27$ for $k = 3$, confirming that A2 is feasible exclusively on small instances, whose primary role is to certify the quality of A1. The problem remains NP-hard despite the convex formulation, since the exponential worst-case complexity is determined by the binary assignment variables $a_{ij} \in \{0, 1\}$.

The GAP analysis on such instances shows that A1 with a single restart reaches the exact optimum in 72% of cases for $k = 2$ and 43% for $k = 3$, with a mean GAP of 4.77% and 16.91% respectively. These results confirm the theoretical finding of Bradley et al. [1]: constrained k -means is less prone to local minima than classical k -means, and a small number of restarts is sufficient to obtain high-quality solutions. The analysis of the effect of the capacity constraint m reveals a fundamental trade-off between cost and balance: as m decreases, the optimal cost increases with respect to unconstrained k -means (up to +23.6% on the medium dataset and +12.6% on hard), but the constraint guarantees more homogeneous clusters. The choice of m therefore allows explicit control over this trade-off depending on the application context.

Limitations and future work. The main limitation of this work is the size of instances for which a certified comparison between A1 and A2 is possible: with $d = 2$, A2 is tractable only for $n \leq 40$ ($k = 2$) or $n \leq 27$ ($k = 3$), making it impossible to verify the GAP on real-sized instances. Future work could explore alternative techniques to extend certifiability to larger instances. On the heuristic side, more sophisticated initialisation strategies could further reduce the number of restarts needed without increasing the computational cost per restart.

References

- [1] P. S. Bradley, K. P. Bennett, and A. Demiriz, “Constrained k-means clustering,” Tech. Rep. MSR-TR-2000-65, Microsoft Research, 2000.
- [2] D. Arthur and S. Vassilvitskii, “k-means++: The advantages of careful seeding,” in *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 1027–1035, 2007.
- [3] D. Aloise, A. Deshpande, P. Hansen, and P. Popat, “Np-hardness of euclidean sum-of-squares clustering,” *Machine Learning*, vol. 75, no. 2, pp. 245–248, 2009.
- [4] A. Frangioni, *Optimization & Learning – Lecture Notes*. University of Pisa, 2025. Version 0.0.1.
- [5] Wikipedia contributors, “Heuristic.” <https://en.wikipedia.org/wiki/Heuristic>, 2025. [Accessed: 2025].
- [6] F. S. Hillier and G. J. Lieberman, *Introduction to Operations Research*. McGraw-Hill, 7th ed., 2001.
- [7] Gurobi Optimization, LLC, “Gurobi Optimizer Reference Manual,” 2023.
- [8] S. P. Lloyd, “Least squares quantization in PCM,” *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–136, 1982.
- [9] Google LLC, “OR-Tools,” 2023.
- [10] A. V. Goldberg and R. E. Tarjan, “Finding minimum-cost circulations by successive approximation,” *Mathematics of Operations Research*, vol. 15, no. 3, pp. 430–466, 1990.